Assignment 5 — One-Dimensional Arrays

Part 1: Multiple Choice (8 Questions · 8 points)

1. For the declaration int scores[8];, which indices are valid?

- ☐ A) 1 through 8
- ☐ B) 0 through 8
- ☐ C) 0 through 7
- ☐ D) 0 through 9

2. An int array starts at base address 1000 and each int takes 4 bytes. At what address is arr[5]?

- ☐ A) 1005
- ☐ B) 1016
- ☐ C) 1020
- ☐ D) 1024

3. Given: int a[5] = {2, 4}; What are the values of a[1] and a[3]?

- ☐ A) 4 and 0
- ☐ B) 4 and garbage (unknown)
- ☐ C) 2 and 4
- ☐ D) It does not compile

4. Which declaration is valid standard C++ (not a non-standard variable-length array)?

- ☐ A) int n; cin >> n; int arr[n];
- ☐ B) int size = 10; int arr[size];
- ☐ C) const int N = 10; int arr[N];
- ☐ D) int arr[];

5. What happens if your code writes to arr[10] when arr was declared as int arr[10];?

- ☐ A) A guaranteed compiler error
- ☐ B) The array grows by one element
- ☐ C) A guaranteed clear crash
- ☐ D) Compiles, but undefined behavior

6. Given: int n[5] = {4, 9, 15, 22, 7}; int c = 0; for (int i = 0; i < 5; i++) { if (n[i] > 8) c++; } What is c?

- ☐ A) 2
- ☐ B) 3
- ☐ C) 4
- ☐ D) 5

7. Which for loop header correctly visits an array of size SIZE from the last element to the first?

- ☐ A) for (int i = SIZE; i > 0; i--)
- ☐ B) for (int i = SIZE - 1; i >= 0; i--)
- ☐ C) for (int i = SIZE - 1; i > 0; i--)
- ☐ D) for (int i = 0; i < SIZE; i--)

8. Which statement about passing an array to a function is TRUE?

- ☐ A) A copy is made; original is safe
- ☐ B) Can change original; const blocks it
- ☐ C) The size is passed automatically
- ☐ D) Call it as printArray(scores[], 5)

Part 2: Coding Problems (22 points)

Problem 1: Test Score Statistics (7 pts)

Write a C++ program called score_stats.cpp that reads 5 test scores into an array and reports some statistics about them.

- ☐ ~~Declare const int SIZE = 5; and an int array of that size.~~
- ☐ ~~Use a for loop to read the 5 scores from the user with cin.~~

- [ ] Use loops to compute the sum, the highest score, and the lowest score (start your max and min from the first element).
- [ ] Compute the average as a double — use static_cast<double> so the decimal part isn't lost.
- [ ] Print all the scores on one line, then the sum, average, highest, and lowest, each clearly labeled.

Example interaction:

```
Enter score 1: 90
Enter score 2: 80
Enter score 3: 70
Enter score 4: 60
Enter score 5: 51
Scores: 90 80 70 60 51
Sum: 351
Average: 70.2
Highest: 90
Lowest: 51
```

Problem 2: Search & Count (7 pts)

Write a C++ program called search_count.cpp that searches a fixed array for a number the user chooses.

- [ ] Start with exactly this array: int data[10] = {7, 12, 5, 12, 40, 3, 12, 8, 21, 5}; (use const int SIZE = 10;).
- [ ] Ask the user for a target number using cin.
- [ ] Use a sentinel (foundIndex = -1) to record the index of the FIRST match, and a counter to track how many times the target appears.
- [ ] If found, print the first index and the number of occurrences. If not, print that it was not found.

Example interactions:

```
Enter a number to search for: 12
12 first found at index 1
12 appears 3 time(s)

Enter a number to search for: 99
99 was not found.
```

Think about how to make sure foundIndex only gets set ONCE, on the first match — not overwritten by later matches.

Problem 3: Array Utilities with Functions (8pts)

Write a C++ program called array_utils.cpp that uses three functions to work with an array passed in from main.

- [ ] In main, create: const int SIZE = 6; int values[SIZE] = {14, 3, 27, 9, 50, 21};
- [ ] Write void printForward(const int arr[], int size) — prints the elements in order.
- [ ] Write void printReverse(const int arr[], int size) — prints the elements from last to first.

- ~~☐ Write int countAbove(const int arr[], int size, int limit) — returns how many elements are greater than limit. main asks the user for the limit and prints the result.~~
- ~~☐ Every function must use its size parameter for its loop bound — no hardcoded 6 inside the functions.~~

Example interaction:

```
Forward: 14 3 27 9 50 21
Reverse: 21 50 9 27 3 14
Count values above: 20
Count: 3
```

Keep all three at the Lesson 5 level: one-dimensional arrays with fixed (const) sizes, plus loops and simple functions. No vectors, no 2D arrays, and no sorting are needed.

Before You Submit

- ☐ All three programs compile and run without errors or warnings on your machine.
- ☐ Files are named score_stats.cpp, search_count.cpp, and array_utils.cpp.
- ☐ A header comment at the top of each file with your name and “Assignment 5.”
- ☐ Each program tested with at least two different inputs, covering different cases (e.g., a target that IS in the array and one that is NOT).
- ☐ No loop reads or writes outside the valid index range 0 to SIZE − 1.
- ☐ explain every line by using backlash //